第8章　目的コード生成

３番地コードとして生成された命令列は３番地コードを解釈する実行系で実行可能である．３番地コードは計算機のアーキテクチャからは独立した命令であり，変数はメモリ上にあるとして実行される．コンパイラにおける目的コード生成は，中間コードに対応する目的計算機の命令語を選択する．ここで，現在の計算機の多くのアーキテクチャでは，メモリにあるデータを演算器のレジスタにロードして演算を行い，レジスタに得られた結果をメモリに書き戻す動作をする．演算器のレジスタとメモリの間のデータの交換速度は一般に計算機の演算に比べて遅いため，プログラムの解析によってレジスタとメモリの間のデータ交換ができるだけ少なくなるような目的コードを生成することが望ましい．中間コードの変数の数に比べてレジスタの数は非常に少ないため，実行プログラムにおいてその時点で頻繁に演算に用いられる変数をレジスタに置くようなコードを生成することによって実行速度の大幅な改善が可能になる．変数とレジスタの対応を決定することをレジスタ割当という．

一般にプログラムに対して最適なレジスタ割当を行うことは難しく，３番地コードのフローグラフが与えられている時に割当アルゴリズムが NP 完全な問題である [?] ため，目的コード生成でのレジスタ割当は最適性を保証することは困難である．従って，目的コード生成では，部分的なプログラムに対して必要なレジスタ数を見積もり，近似的に効率のよい割当手法を与える．具体的には，３番地コードや SSA 形式における変数のデータフローからライフサイクル（生存期間）を解析して，変数からレジスタへの割当を決定する．

目的コードの命令の選択については目的マシンのアーキテクチャに応じてできるだけサイクル数の少ない命令（軽い命令）を選択するために命令のコストを見積もるなどの手法がある．しかし，実際の計算機ではハードウェアによる並列的な実行などの効果や実行ユニットの数などに依存することから，実際的な計算機に対するコード生成の系統的なモデル化は教科書的内容の範囲を越えるため，本書では扱わない．

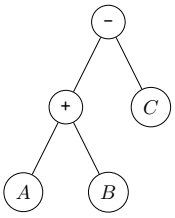
8.1　算術式のコード生成

ここでは，算術式に対するコード生成において必要なレジスタの数を見積もる手法を示す．簡単化のため，以下の範囲に限定する．

- 演算子は四則演算 $(+, -, *, /)$ に限定する．
- 命令はメモリ転送命令 $(\texttt{Load}\ r, e)$ と演算命令 $(\theta\ r, e)$ とする．
  メモリ転送命令 $\texttt{Load}\ r, e$ は，e で示されるメモリの内容をレジスタ r に転送する．演算命令 $\theta\ r, e$ は，e で示されるメモリ（またはレジスタ）の内容をフェッチして r と $\theta \in \{+, -, *, /\}$ で定められた演算を行い，結果を r に格納する．

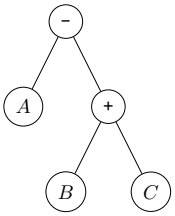
例えば，算術式 $A+B-C$ のコードは以下のようになる．

```
Load  r1  A
+     r1  B
-     r1  C
```



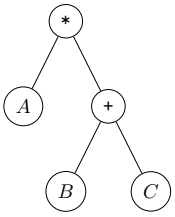
以上の制約のもとでは，レジスタは 1 個で計算が可能である．これに対して，$A-(B+C)$ の場合は以下のような命令系列になる．

```
Load  r2  B
+     r2  C
Load  r1  A
-     r1  r2
```



この算術式の計算に必要なレジスタ数は 2 である．$A*(B+C)$ については，

```
Load  r2  B
+     r2  C
*     r2  A
```



とすればよいので必要なレジスタは 1 つである．これは，$-$ が可換でなく，$*$ が可換であるため 2 番めの項をまず評価して 1 番目の項を適用することができるためである．算術式を表す抽象構文木 T に対して，その計算に必要なレジスタ数を $RN(T)$ と書く．$RN(T)$ はアルゴリズム 1 で求められる．

このレジスタ数の見積もり解析に応じて，使用するレジスタが実際に利用できるレジスタ数の範囲内であれば，レジスタを利用する目的コードを生成できる．しかし，レジスタ数が足りない場合は，レジスタ溢れ (spill) が発生する．このような場合は，フロー解析などの手法を使って最も後で使われる変数をレジスタに割り当てるなどの方法によって効率的に実行する目的コードを生成する．

8.2　グラフ彩色によるレジスタ割当

3 番地コードの基本ブロックに対して計算に用いられる変数をできる限り多くレジスタに割り当てる方法について紹介する．基本的なアイデアとして変数の生存期間，すなわち，変数が定義された時点からその定義最後に使われるまでの期間，が重ならない 2 つの変数は同じレジスタに割り当てることができるという性質を利用する．

Algorithm 1 $RN(T)$ の計算

```
procedure RN(T)
    if isleaf(T) thenx return 0                                  ▷ T が葉
    else
        T_1 = Left(T)                                            ▷ T_1 は左部分木
        T_2 = Right(T)                                           ▷ T_2 は右部分木
        if RN(T_1) = 0 かつ RN(T_2) = 0 then return 1    ▷ 左右とも葉なのでレジスタ１つ
        else
            if RN(T_1) = 0 かつ RN(T_2) > 0 then
                if root(T) が可換演算子 then return RN(T_2)          ▷ 後ろから適用する
                else return max(2, RN(T_2))                      ▷ 最大２つ必要
                end if
            else
                if RN(T_1) > 0 then
                    if RN(T_1) = RN(T_2) then return RN(T_1) + 1
                    else return max(RN(T_1), RN(T_2))
                    end if
                end if
            end if
        end if
    end if
end procedure
```

8.2.1　レジスタ干渉グラフ

レジスタ干渉グラフは，生存期間の重なりを無向グラフによって表現したものである．無向グラフの頂点に変数を割り当ててその生存期間が重なる場合には，その頂点間に辺をつける．互いに生存期間が重ならない変数については対応する頂点は辺を持たない．従って，$(u, v) \notin E$ のとき，u に割り当てられた変数と v に割り当てられた変数は同じレジスタに割り当てることができる．

例 8.1.

無向グラフに対する k-彩色問題とは，辺の両端にある頂点に違う色を割り当てるときに k 種類の色によって塗り分けが可能かどうかを判定する問題である．

定義 8.1. 無向グラフ (V, E) に対する k-彩色問題とは，すべての辺 $(u, v) \in E$ にたいして，$f(u) \neq f(v)$ となる関数 $f{:}V \to \{1, \ldots, k\}$ が存在するとき，*YES,* 存在しないとき *NO* と答える問題である．

レジスタ干渉グラフを構成すると k 彩色問題で YES となる関数 f は，レジスタ割り当てそのものである．グラフの彩色問題は NP 完全な問題であり，一般に最適なレジスタ割り当てを求めることはコンパイラの構成として実際的ではない．

実際的なレジスタ割り当てを行うために以下の定理を利用することができる．

定理 8.1. 無向グラフ (V, E) の任意の頂点の次数が $k - 1$ 以下ならば k-彩色可能である．ここで，$k \geqslant 1$ とする．

この定理を利用してレジスタ割り当てアルゴリズム**??**が構成できる．

このアルゴリズムにおいて，定理 8.1 の条件が成り立たない時，要素数最大の独立集合を求める必要がある．要素数最大の独立集合を求める問題は NP 完全問題であり，このアルゴリズムをそのまま実現することは実用的とは言えない．

8.2.2　SSA に対するレジスタ割当

φ 関数を消去して得られる SSA 形式からは，比較的容易にレジスタ割り当てを導くことが知られている．SSA 形式では，最適化後において支配関係が保存されている．従って，各変数の生存期間は定義において新たな添字をもつ変数が定義されてから，次の添字を持つ変数が定義されるまでの間となる．各変数に対して生存期間に対応する頂点を抽出し，その頂点から構成される支配木の部分木を構成する．その部分木の交差グラフを構成するとレジスタ干渉グラフが構成できる．

補題 8.1. *SSA* 形式から支配木の部分木から構成された交差グラフによる干渉グラフは弦グラフである．

弦グラフの頂点に対しては，完全消去順序が存在する．完全消去順序の逆順で彩色をすると最小の数の色でグラフを彩色することができる．SSA 形式による干渉グラフの頂点数を n, 辺数を m とするとグラフ構築後の最適彩色は $O(m + n)$ で構成できる．

ここでは，SSA から構成される干渉グラフの大きさに対して線形時間で最小の色数で彩色できる．ただし，最初から色数を指定して彩色する問題ではないため，レジスタ溢れについての問題を

解いたわけではないことに注意する．SSA 形式からのレジスタ割り当ては，生存解析が容易になることによって効率的で最適な割り当て手法を与える．レジスタ数が少ない場合のレジスタ溢れといった問題は，依然，計算量の大きい問題として残ることに注意しよう．